

Project Proposal: Black-Box Evolutionary Attacks on Growing, Regenerating Neural Cellular Automata

NC team 28

April 10, 2026

Todo list

Review Planning	4
list optional experiments as examples	4

1 Context

Write a 2-page project proposal, in pdf, with the following elements: – A (tentative) title for your project – A problem statement with the background, relevance, and a concrete description of the problem you are solving (use exercise B.1 as input) – A section "related work" describing the relation of your project to other work in this area (use exercise B.2 as input) – A section "Approach" describing which methods you will use, why they are appropriate, which resources you need, and where/how you will obtain those (use exercise B.3) – A section "Planning" where you describe which milestones you want to finish when (use exercise B.4). Note that many of these elements will likely end up in your final report as well, so it is worth investing some time in this! Your final proposal should be concrete enough that it allows you to get started on the project. Hand in your proposal on Brightspace per the deadline listed there.

improved first pitch draft:

Neural Cellular Automata (NCAs) are small shared-weight networks that grow images from a single seed pixel and heal themselves after damage, and are often presented as very robust. Randazzo et al. (2021) and Cavuoti et al. (2022) showed you can still reprogram a trained Growing NCA, for instance making the target lizard lose its tail or turn red, using white-box, gradient-based attackers that have full access to the network's weights and backpropagate through its rollout.

Every published NCA attack uses gradients. Every published NCA robustness claim is correspondingly based on a stronger and less "natural" threat model than what an adversary in, say, a biological or bioengineering setting would plausibly have. We ask whether a query-only, black-box attacker using evolutionary search can reproduce the same targeted reprogramming attacks, and how close it gets to the gradient-based ceiling.

Concretely, we do not modify the NCA's weights and we do not perturb pixels directly. Following Randazzo et al., the attack surface is a 16×16 state-perturbation matrix, applied identically to every cell's 16-dimensional state vector (4 visible RGBA channels plus 12 hidden communication channels) at every timestep of the simulation. A 64×64 grid therefore has roughly 65,000 state values being transformed by the same 256 attack parameters each step. This compact, attack surface is exactly the type where CMA-ES could perform well (which doesn't perform well over 1000 parameters).

Our attacker sees only the visible RGB output at the end of the rollout, but never the weights, never the hidden channels, never any gradient information. Fitness is the L2 distance between that final image and a specific target image (e.g. a tailless lizard), so the attacker has to steer the NCA toward a chosen broken configuration. This matches Randazzo et al.'s targeted setup and makes the comparison of optimizers possible and also compare with random noise.

If evolutionary search can meaningfully reprogram the NCA under this restricted threat model, the robustness of Growing NCAs could be weaker than the literature implies. If it can't, the gap between

white-box and black-box attacks on NCAs is a nice safety margin and shows they are robust against more 'realistic, or natural' attacks.

Things we can/should do if comparing attack types/optimizers isn't enough

- Untargeted ("destroy only") attack as a second condition: *fitness* = maximize L2 from clean target, should be easy
- Adversarial training / arms race — take the attacks CMA-ES finds, add them to the damage distribution during NCA training. Does the NCA become robust to evolved attacks? Much bigger scope, only if we really have time
- Show efficiency curves of ES vs Gradient-based (should probably def do this, track it while we go). Compare needed number of queries for each optimizer etc.
- Try more than the default target image, easy
- see if perturbations persist or get replaced with time once we stop attacking, easy

2 Actual paper starts here, put your stuff:

Problem Statement

Neural Cellular Automata (NCAs) are small, shared-weight networks that grow images from a single seed pixel and can regenerate after damage, making them a promising model for robust, self-organizing systems. Their perceived robustness has led to NCAs being deployed as a defensive component in larger architectures: Xu et al. [2] use NCA layers as plug-and-play adaptors inside Vision Transformers to make them more resistant against adversarial samples. Whether NCAs themselves can be subverted is therefore not just of theoretical interest but relevant to the security of systems that rely on them.

Randazzo et al. [3] and Cavuoti et al. [4] demonstrated targeted attacks that, for example, can make a generated lizard image lose its tail or change color. However, every published attack on NCAs relies on white-box, gradient-based methods that require full access to the model’s weights and backpropagation through the rollout. Robustness claims are so far tested only against this strong threat model. Additionally, focusing on the nature-inspired roots of NCAs, a white-box is not what a realistic adversary, for instance in a biological or bioengineering setting, would plausibly have access to.

We therefore ask: *can a query-only, black-box attacker using evolutionary search reproduce the same targeted reprogramming of a Growing NCA, and how close does it get to the gradient-based ceiling?* We attack the NCA via a 16×16 state-perturbation matrix applied to every cell’s 16-dimensional hidden state at every timestep, the same attack surface used by Randazzo et al. Our attacker observes only the final RGB output and optimizes the L2 distance to a chosen target image using CMA-ES, without ever accessing model weights, hidden channels, or gradient information.

If Growing NCAs can be influenced by this attack in a similar way as through white-box approaches, they may be less robust than priorly assumed. Otherwise, they may be safe from more realistic attack types and to be used in public-facing, but not open-weights, ML systems.

Related Work

Mordvintsev et al. [1] introduced the Growing NCA model and showed that, through a sample pool training strategy and exposure to damage during training, the learned local update rules become robust enough to regenerate damaged parts. Due to this robustness, NCAs have also been used defensively: Xu et al. [2] inserted NCA layers between vision transformer blocks to serve as an armor against adversarial attacks.

On the offensive side, Randazzo et al. [3] investigated white-box vulnerabilities of Growing NCAs and showed that, while they are more resistant to local injections, they can still be reprogrammed using global state perturbations applied to every cell at every step. Similarly, Cavuoti et al. [4] demonstrated that injecting a small number of adversarial cells into the grid can drastically alter the grown image. Both studies rely on gradient-based optimization with full access to model weights.

Covariance Matrix Adaptation Evolution Strategy (CMA-ES), introduced by Hansen [5], is an algorithm for non-linear, non-convex black-box optimization that maintains a population of candidates and iteratively shifts them toward high-fitness regions. While evolutionary black-box attacks are commonly used to evaluate traditional image classifiers, they have not yet been applied as an adversarial tool against cellular automata.

Approach

We will first train a regenerating NCA using the PyTorch re-implementation by Greydanus [6]. Using the trained weights, we will apply CMA-ES [5] to optimize the 16×16 state-perturbation matrix described in the Problem Statement, steering the NCA toward a chosen target image (e.g., a red lizard instead of a green one). CMA-ES is well suited here because the perturbation matrix has only 256 parameters, which falls within the range where CMA-ES is known to perform effectively.

We will compare the attacks using the CMA-ES algorithm to a random baseline of the same intensity, and, likely after the prototype phase, to the white-box gradient-based attack from Randazzo et al. [3], whose code is publicly available as a TensorFlow notebook and which we will adapt to our PyTorch setup. We will run qualitative experiments on feasibility, divergence, efficiency curves, and more. Given that we have sufficient time, we will likely conduct additional experiments, such

as destructive fitness goals/loss function of maximizing L2 from clean target, different target images, whether the perturbations persist after the attack ends, and perhaps even if we can boost the robustness of the NCA further by training it to resist evolved attacks.

In terms of resources, this project is fairly lightweight by neural ML standards and should work with Google Colab or personal machines, which we readily have access to. We can use the existing PyTorch NCA implementation by Greydanus and the target images from Mordvintsev et al.'s paper. For the gradient-based baseline, Randazzo et al. have shared their attack code as a TensorFlow notebook, which we will port to PyTorch to keep everything in a single framework.

Review Planning

Planning

To ensure steady progress and meet the project deadlines, we have structured our timeline into weekly milestones.

- **April 10 – April 15; Basis:** Set up the development environment and the Greydanus PyTorch NCA implementation. Successfully train a standard Growing NCA on a target image (e.g., the lizard), verify its self-organizing properties, and save the model weights.
- **April 15 – April 20; Prototyping:** Implement the attack surface logic (the 16×16 state-perturbation matrix). Implement the "Random Noise" baseline attacker. Integrate a basic CMA-ES loop to prove the end-to-end pipeline works (from perturbation generation $\rightarrow$ NCA rollout $\rightarrow$ L2 fitness evaluation).

Milestone: Working Prototype comparing Random Noise by April 20.

- **April 20 – April 27; Gradient Baseline & Tuning:** Port the white-box gradient-based attack from Randazzo et al.'s publicly available TensorFlow notebook to our PyTorch setup. Concurrently, tune the CMA-ES hyperparameters (population size, initial variance) for optimal targeted attack performance.
- **April 27 – May 22; Experiments & Data Collection:** Run the core comparative experiments across all three conditions (CMA-ES vs. Gradient vs. Random). Extend the experiments to include those in the optional list (e.g., ...).

list optional experiments as examples

Collect quantitative data, specifically tracking efficiency curves (number of queries/forward passes versus the resulting L2 distance from the targeted "broken" state).

- **May 22 – June 5; Finalizing Experiments & Paper Drafting:** Continue with list of Experiments. Draft the Results, Discussion, and Conclusion sections of the paper.

Milestone: Project reaching maturity with a first draft by May 29.

- **June 5 – June 12; Finalizing:** Format the final paper, review against the grading criteria, and submit.

References

- [1] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, “Growing neural cellular automata,” *Distill*, vol. 5, no. 2, e23, 2020. [Online]. Available: <https://distill.pub/2020/growing-ca> doi: 10.23915/distill.00023.
- [2] Y. Xu, T. Zhang, and S. Süssstrunk, “AdaNCA: Neural cellular automata as adaptors for more robust vision transformer,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 25709–25746, 2024.
- [3] E. Randazzo, A. Mordvintsev, E. Niklasson, and M. Levin, “Adversarial reprogramming of neural cellular automata,” *Distill*, vol. 6, no. 5, e00027-004, 2021. [Online]. Available: <https://distill.pub/selforg/2021/adversarial> doi: 10.23915/distill.00027.004.
- [4] L. Cavuoti, F. Sacco, E. Randazzo, and M. Levin, “Adversarial takeover of neural cellular automata,” in *Artificial Life Conference Proceedings 34*, 2022, p. 38. MIT Press.
- [5] N. Hansen and A. Ostermeier, “Completely derandomized self-adaptation in evolution strategies,” *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.
- [6] S. Greydanus, “Studying growth with neural cellular automata,” blog post and PyTorch implementation, 2022. [Online]. Available: <https://greydanus.github.io/2022/05/24/studying-growth>